Testing

Cohort 3 Team 3

Fares Almutairi
Taro Chandiwana
Lily Gaunt
Jacob Last
Alex Ponomarenko
Giulia Sclano

Our Testing Approach

Our overall testing strategy was to focus on achieving meaningful coverage as opposed to maximal coverage, as this is unfeasible. Therefore, we focussed on methods that contained core game logic, as this is the code which is critical to the operation of the game and is likely to fail or get broken by developers.

Testing was at the forefront of our minds from the second we took over this project, and because of this we started writing tests and set up automated testing via Github Actions before we started editing the code. The testing process began with writing tests for the main game file, YettiGame.java and we continued to write tests for the rest of the code at the same time as new code was being developed. This allowed us to catch errors developers made early, before they became a bigger problem.

Types of Test

Our goal was to attempt to stick close to the 80% unit tests, 15% integration tests and 5% manual tests pyramid as discussed in the lecture. Essentially if a unit test is possible in the given situation, then that is what should be used. This avoids flakey tests that result due to top-heavy test antipatterns (e.g. ice cream cone).

Using short unit tests means the tests run quickly and are less likely to be broken as the game is developed. Debugging issues identified by tests like this is also much easier. The requirements refer to broad parts of the system, which is not granular enough to test using this testing strategy. Therefore, we broke them into unit-testable sub-requirements. White box tests can be written for these sub-requirements, providing confidence that the high-level requirements have been met without the need for black box tests.

For complex game logic/methods, we divided the input space into equivalence classes and tested one value from each equivalence class and the boundary values to ensure all cases operate as intended.

Manual tests are also necessary due to aspects of the system such as graphics, UI elements and system requirements being impossible to test without human judgement.

Testing Tools Used

For the unit testing and integration tests, we used the JUnit framework as this is the industry standard for testing Java code. We also used Mockito for the creation of stubs to control method return values. This allows us to reduce the number of flakey tests by removing dependency on multiple parts of the system. Alternatives such as EasyMock offer similar functionality, however Mockito is well-maintained and was the one used in the lectures, so we decided it was most appropriate. We also used JaCoCo to generate a line coverage report.

Testing Statistics

Automated Tests

Test Report Links

In order to generate a test report for our automated tests, we ran them using the command ‘./gradlew test’ to automatically generate a HTML report https://eng1-group3.github.io/website-2/public/Testing%20Material/junit/testing%20statistics.html. This report shows a detailed report of test runtimes and which tests passed / failed.

In order to demonstrate the completeness of our tests we decided to produce a table that maps all of the requirements of our system to the specific tests that were executed, so that we can verify that each requirement maps to at least one automated or manual test. The mapping of all requirements to automated tests can be found here: https://eng1-group3.github.io/website-2/public/Testing%20Material/Requirements%20to%20Automated%20Tests%20Mapping.pdf.

Our line coverage report can be found here: https://eng1-group3.github.io/website-2/public/Testing%20Material/jacoco/html/index.html. This should be taken with a pinch of salt, as the main areas missing automated tests are the ‘screens’, the area of the project, which use logic from other parts of the game and outputs it to the screen (hence very difficult to test using Junit). Therefore, as long as we test the game logic works, these classes will work as intended. Output to the user is mainly tested in our manual tests. As you can see, we achieved 70% and 82% test line coverage on the main package and the entities package respectively, which really supports the fact that our game logic works as intended.

Our Automated Tests

In our final build we had 128 JUnit tests, of which 126 passed. The two tests that failed are discussed below, in the ‘Input-Space Coverage Example’ section below.

Although some of these tests utilise multiple classes and test how they work together, all of these are unit tests, and none are integration tests. This is mainly due to LibGDX dependencies such as rendering to the screen making integration testing difficult - mocking the dependencies is a much more reliable option and avoids complex setup required for a full headless backend.

This is obviously a lot more unit-test heavy than what we were aiming for, however this still provides good test coverage of the core functionality of the system. The less dependencies each test has on other parts of the system, the easier it is to debug when a test fails, as narrow-scoped tests mean the error is more isolated. This makes it much easier to identify the root cause of what is making the system not work as intended.

Another reason why we didn’t have as many integration tests as we were originally aiming for is due to us deciding that instead of utilising lots of parts of the system in a single test, it would be more beneficial to use mockito to simulate classes and then just write a separate unit test for the method that is being mocked. This way both methods get tested and they

don’t depend on each other in order to pass, making it easier to pinpoint the cause if either method fails.

Unit tests

These test small, isolated parts of the system. As already discussed, we used Mockito for these, allowing our tests to be more concentrated on individual methods rather than testing a larger part of the system.

We attempted to write our unit tests in a systematic order, going through all of the Java files in the game folder and creating a file of the same name + ‘Test’ for each one. We then went through the classes’ methods, and attempted to write at least one unit test for almost every method that contained non-simple logic and was possible to unit test (i.e. doesn’t attempt to render anything to the screen or contain too many LibGDX dependencies).

If applicable, we wrote multiple unit tests for each method, attempting to achieve as much line coverage as possible by testing the method with a variety of input equivalence classes and boundary values in order to test a representative for every case that may occur when the game is actually run. This ensures methods have each logical branch tested, without the need to use an automated line coverage tool such as JaCoCo in order to generate quantitative line coverage statistics.

Input-Space Coverage Example

A great example of this is the unit tests for MapManager. This class has methods that determine whether a given position or rectangle is in a valid place in the map (i.e. not overlapping with a wall). Despite only testing two methods, there are 14 unit tests in order to verify this logic. The first 5 tests verify the method that checks if a given coordinate is an invalid position in the map. The cases for these 5 tests are: a valid coordinate, a coordinate in a blocked cell, a coordinate outside of the map and two boundary coordinates either side of the blocked cell boundary. This ensures a representative from all input equivalence classes is covered.

The second method tested in this class determines whether a rectangle is in an invalid position of the map, i.e. if it collides partially or fully with a wall. The 9 test cases (in the order they are in MapManagerTest.java) are as follows:

1. A rectangle fully in the blocked cell,
2. A rectangle overlapping with the blocked cell (one of its corners in the blocked cell)
3. A rectangle that is entirely outside of the map
4. A rectangle that is part inside and part outside of the map
5. A rectangle that only just collides with the blocked cell (invalid side of the boundary)
6. A rectangle in a valid position
7. A rectangle that only just avoids the blocked cell (valid side of the boundary)
8. A rectangle that has the blocked cell entirely within it
9. A rectangle where one of its sides crosses over the blocked cell (such that none of its corners are within the blocked cell)

There are definitely even more cases that could be covered, such as testing the left, right, top and bottom boundaries of the blocked cell individually. However, these 9 tests cover a

large proportion of the possible input equivalence classes, such that if these 9 tests pass it is almost guaranteed that all cases that can be given to this method work as intended.

However, after writing these 9 cases, we discovered an error with the MapManager code. Upon closer inspection of the MapManager code this is because isRectInvalid() works by verifying if any of the rectangle's corners are invalid. If any of them are, then the rectangle is also said to be invalid. Therefore, tests 8 and 9 fail, as in these cases all of the rectangles corners are valid, meaning isRectInvalid() returns false, even though the rectangle actually does overlap with the blocked cell.
After liaising with the development team, we concluded that this was not game-breaking for the current implementation of the game, as all components that move and have hitboxes have smaller hitboxes than the walls of the maze, therefore this situation can never occur in a real-game environment.

If an obstacle with a smaller hitbox were to be added, the code for MapManager would have to be fixed so that it passes these test cases, hence why we kept them in as failing tests to help guide future development. In order to pass test 8, the method would need to be coded to utilise the x & y coordinates of the corners or the rectangle in order to check if any of the four edges overlap with the blocked cell. For test 9, the method would need to use the x & y coordinates of the corners of the blocked cell in order to test if they are inside of the rectangle. If this is implemented, then the method will pass all tests.

Manual Tests

In order to create our manual tests we looked at the table that mapped requirements to automated tests (linked again here: https://eng1-group3.github.io/website-2/public/Testing%20Material/Requirements%20to%20Automated%20Tests%20Mapping.pdf).

Underneath this table, we have noted down all requirements that have not been tested by an automated test. We concluded that these are the requirements that we should prioritise to target when creating our manual tests.

For the conciseness of this document, the manual tests can be found here: https://eng1-group3.github.io/website-2/public/Testing%20Material/Manual%20Tests.pdf.

In this document, there are 8 manual tests. Each test has an overall summary, a list of the requirements it is verifying we have met, and the steps required to carry out the test. The expected result is given followed by the actual result of what happens if you perform the test on the final version of our game. All 8 of these manual tests pass.

Conclusion

Together, these two external documents show that every requirement has been tested. From the table it can easily be seen that every requirement that is not tested with an automated test has a manual test in order to test it, therefore all requirements have been tested.